

OpenVid Video Chunker: Memory Pile-Up Analysis

cantina-tools / Geneva quickstart | 2026-06-09

Summary

The OpenVid chunking refresh drove worker RAM upward without bound, concentrating on a handful of nodes while the rest sat idle and the GPUs went unused. The cause is structural, not a leak: a Geneva chunker-backed materialized view executes each work item by **buffering its entire expanded output in memory on a single actor before returning anything**. With a full-resolution PNG stored on every one-second clip, that per-actor buffer grew into the tens of gigabytes. A stalled refresh (29 minutes, zero rows produced) confirmed actors were accumulating output and never completing a batch. The fix shrinks the per-row payload and the per-clip decode cost; the orphaned compute was stopped on both the Geneva and Ray layers.

Observed Symptoms

- RAM heavily skewed: a few workers at 12–31 GB, most near 0.8 GB; GPUs 0% utilized (the work is CPU-only decode/remux).
- The running refresh job reported `batches 0/10` and `rows_produced 0/0` after ~29 minutes — 10 actors busy, none having finished a single work item.
- Dozens of terminal Ray *client-server driver* jobs lingered from prior connections, the documented failure mode where a cancelled Geneva job leaves its driver (and actors) holding RAM.

Root Cause

The view is a **chunker materialized view** (`create_udtf_view + @chunker`). Its refresh runs through Geneva's `_append_expanded_fragments` path, where:

- Work is split into items of up to **1024 source rows** (`row_id_batch_size`, hard-coded). The number of parallel actors is `min(ceil(videos/1024), concurrency)` — so ~10k videos yields 10 actors and `concurrency=48` is capped to 10 by the data.
- Each actor's `expand_batch` **materializes the full expansion for its whole work item** (up to 1024 videos × clips/video × bytes/clip) as one in-memory Arrow table, then ships it back over IPC. This buffer is the pile-up.
- `max_rows_per_fragment` (our `checkpoint_size`) is applied *on the driver afterward*, only to slice the finished result into output fragments. **It does not bound actor memory** — correcting an earlier assumption.
- The dominant per-row term was a **full-resolution lossless PNG** of each clip's first frame. Downstream, CLIP/BLIP downsize to ≤336 px and OpenPose rescales internally, so that resolution was largely wasted.
- Levers that do **not** help: `partition_by` (ignored on the chunker path) and source-table fragment size set by `ingest` (all new row-ids are re-batched into 1024-row work items regardless).

Changes Applied

Action	Change	Effect
Stop compute	Cancelled the running Geneva refresh; killed the orphaned Ray client-server driver on the head. Verified 0 active on both layers.	Frees actors/RAM held by the stalled job and stale drivers.
Frame payload	Store a 512 px JPEG (q85, longest side, LANCZOS) instead of a full-res PNG. Column stays <code>large_binary</code> ; consumers use <code>Image.open()</code> , so no schema or downstream change.	~10–30× smaller per-row frame — the main lever on per-actor buffer size. Near-lossless for CLIP/BLIP/OpenPose.
Per-clip decode	Open each clip's container once : decode the start frame and remux the [start, end) packets from the same handle (was two opens + two full-byte copies per clip).	Halves per-clip decode setup and transient byte-buffer churn.

Fragment size	checkpoint_size 256 → 32.	Healthier output fragments and smaller driver-side commits (secondary; not the memory fix).
Concurrency	Left at 48 (per request).	Effective parallelism is bounded by work-item count (~10), not this value, for the current dataset.

Expected Outcome & Validation

With the frame reduced to a 512 px JPEG, the dominant per-row payload becomes the one-second stream-copied clip (tens to hundreds of KB), so each actor's buffered expansion table shrinks by roughly an order of magnitude. On the next refresh, confirm success by watching `rows_produced` advance steadily and per-worker RAM stay flat rather than climb.

If deeper parallelism or a hard memory ceiling is needed later: the 1024-row work-item size is fixed in this Geneva build, so the remaining options are (a) refresh in smaller source waves to cap videos-per-item at the cost of fewer concurrent actors, or (b) raise the work-item count by scaling total video volume. Neither is required to resolve the current pile-up.